

SHiPGeoModel Simulation Documentation

August 3, 2026

Contents

1 Overview	2
2 Project Structure	2
3 Configuration System	2
4 Core Classes	2
4.1 RunConfig	2
5 main_g4.cpp	3
6 DetectorConstruction	3
7 CalorimeterBuilder	4
8 Fibre_HPLayer	4
9 Scintillator Layers	5
9.1 PVTBarLayer	5
9.2 FSBarLayer	5
10 Materials	5
11 Visualization	6
12 Geometry Database	6
13 Simulation Flow Summary	6
14 Design Philosophy	6
15 Extending the Simulation	7

1 Overview

SHiPGeoModel is a modular detector geometry and simulation framework built using:

- **GeoModel** for detector geometry description
- **Geant4** for particle transport and physics simulation

The framework provides:

- Modular calorimeter layer construction
- Configurable simulation control
- Optional visualization
- Support for rotated fibre high-precision layers
- Separation of geometry and simulation logic

2 Project Structure

include/	Header files
src/	Implementation files
main_g4.cpp	Geant4 simulation entry point
RunConfig.*	Configuration parsing
geometry.db	Generated GeoModel database
calo.cfg	Example run configuration

3 Configuration System

Simulation parameters are defined in a configuration file (e.g. `run.cfg`).

Example:

```
n_events=100
macro=../run.mac
visualize=1
vis_macro=../vis.mac
```

These values are parsed by the `RunConfig` class.

4 Core Classes

4.1 RunConfig

Files:

- `include/RunConfig.hh`
- `src/RunConfig.cc`

Purpose

Stores runtime configuration parameters parsed from a configuration file.

Main Members

```
int n_events;  
std::string macro;  
bool visualize;  
std::string vis_macro;
```

Responsibilities

- Parse key-value pairs
- Provide typed access to parameters
- Control visualization and execution mode

5 main_g4.cpp

Purpose

Entry point of the Geant4 simulation.

Execution Flow

1. Read configuration
2. Construct `G4RunManager`
3. Initialize geometry via `DetectorConstruction`
4. Initialize physics list
5. Optionally start visualization
6. Execute macro or run events

Visualization is activated if:

```
visualize=1
```

6 DetectorConstruction

Files:

- `include/DetectorConstruction.hh`
- `src/DetectorConstruction.cc`

Purpose

Defines the full detector geometry for Geant4.

Responsibilities

- Construct world volume
- Convert GeoModel geometry to Geant4
- Apply visual attributes
- Register sensitive detectors

Key Method

```
G4VPhysicalVolume* Construct();
```

7 CalorimeterBuilder

File:

- src/CalorimeterBuilder.cpp

Purpose

Builds the calorimeter stack according to configuration layer codes.

Layer Codes

Code	Description
1	Wide PVT scintillator layer (along Y)
2	Wide PVT scintillator layer rotated 90° (along X)
3	Thin PS scintillator layer (along Y)
4	Thin PS scintillator layer rotated 90° (along X)
5	Fibre HPLayer (fibres along Y)
6	Fibre HPLayer rotated 90° (fibres along X)
7	Lead absorber (Iron for the HCAL)
8	Air gap

Table 1: Layer codes used in the detector geometry.

Responsibilities

- Compute total stack thickness
- Place layers sequentially along Z
- Call appropriate builder classes

8 Fibre_HPLayer

Files:

- include/Fibre_HPLayer.hh
- src/Fibre_HPLayer.cpp

Purpose

Constructs high-precision fibre layers inside aluminium casings.

Build Interface

```
static void build(  
    GeoVPhysVol* mother,  
    GeoMaterial* aluminumMat,  
    GeoMaterial* fiberMat,  
    double zCenter_mm,  
    int layerIndex,  
    double casingXY_mm,  
    double casingZ_mm,  
    double fiberDiam_mm,  
    bool fibresAlongY = true  
);
```

Orientation Handling

- `fibresAlongY = true` → fibres run along Y
- `fibresAlongY = false` → fibres run along X

Internally, fibres are constructed as cylinders aligned along Z and rotated:

- `RotateX(90°)` converts Z-axis to Y
- `RotateY(-90°)` converts Z-axis to X

9 Scintillator Layers

9.1 PVTBarLayer

Constructs wide plastic scintillator bar layers arranged in a grid.

9.2 FSBarLayer

Constructs thin sampling scintillator layers.

10 Materials

Materials are managed centrally and include:

- Aluminium
- Polystyrene (fibre material)
- Lead
- Iron

11 Visualization

Visualization is controlled via configuration:

```
visualize=1
vis_macro=../vis.mac
```

Example macro:

```
/vis/open OGL
/vis/viewer/set/style surface
/vis/drawVolume
/vis/scene/add/axes 0 0 0 1 m
```

Visual attributes (color, transparency) are assigned in Geant4 after geometry construction.

12 Geometry Database

The GeoModel database can be generated via:

```
./make_leadplate_db
gmex geometry.db
```

This allows geometry inspection independent of Geant4 simulation.

13 Simulation Flow Summary

```
readRunConfigFile()
  ↓
Construct RunManager
  ↓
DetectorConstruction builds geometry
  ↓
CalorimeterBuilder places layers
  ↓
Initialize physics
  ↓
Optional visualization
  ↓
BeamOn / macro execution
```

14 Design Philosophy

- Modular geometry components
- Configuration-driven simulation
- Clear separation between geometry and run control
- Support for rotated fibre layers
- Extendable layer code system

15 Extending the Simulation

To add a new layer:

1. Define a new layer code
2. Extend `CalorimeterBuilder`
3. Implement a builder class
4. Update thickness calculations
5. Optionally define visual attributes